

Data Warehouse Design Report

Movie Rental Analytics System (Based on Sakila Database)

1. Introduction

This report presents a high-level Data Warehouse (DW) architecture designed for a movie rental enterprise, developed systematically from the operational **Sakila OLTP** database schema. While operational systems (OLTP) excel at handling rapid day-to-day transactions, updates, and consistent record inserts, they heavily rely on full normalization to eliminate data redundancy. This creates a dense web of dozens of interconnected tables that perform poorly under heavy corporate analytic reporting.

Conversely, a Data Warehouse employs Dimensional Modeling principles to flatten out structural anomalies and group transactional entities into descriptive, denormalized dimensions surrounding core transactional fact tables. This design dramatically accelerates query performance, minimizes complex multidimensional execution times, and offers an integrated, intuitive environment for decision-makers and data analysts to track high-level trends.

2. Business Questions

To ensure the warehouse delivers direct institutional value, a set of crucial operational and analytical business questions has been formulated to guide the model design:

- **Financial Performance & Temporal Analysis:** Which operational time periods (days, months, fiscal quarters, seasonal periods) demonstrate the highest transactional volume and velocity? Which brick-and-mortar store branches generate the highest net profit margin?
- **Customer Lifecycles & Segmentation:** Who are the highest-spending and most frequent rental consumers across different regions? What is the explicit geographical concentration of active customer cohorts?
- **Catalog Performance & Demand Metrics:** Which movie genres and inventory classifications experience the highest relative velocity? Does a higher rental price point (`rental_rate`) statistically depress rental counts, or do top tiers maintain volume?
- **Inventory Velocity & Stagnant Assets:** Which movie titles show high copy availability across store inventories but suffer from exceptionally low turnover rates (dead stock)?

3. Dimensional Model Design

3.1 Selected Business Processes

The system focuses on the two primary core processes that define the operational cycle of the enterprise:

- **The Rental Process:** Captures physical inventory distribution, transactional lifecycles, and movie check-out frequencies.
- **The Payment Process:** Captures the corresponding direct financial inflows, revenues, and monetary realizations.

3.2 Fact Tables

Fact_Rental

- **Grain:** One individual row per discrete rental event of a specific movie copy by a customer at a given point in time.
- **Measures:** `rental_count` (Atomic increment equal to 1 per transaction line), `duration_days` (The exact calculated span of the rental period).
- **Foreign Keys:** `customer_dim_id`, `film_dim_id`, `store_dim_id`, `date_dim_id`, `staff_dim_id`.

Fact_Payment

- **Grain:** One separate row per individual monetary invoice statement or transaction item cleared by a client.
- **Measures:** `amount` (The numeric payment value captured).
- **Foreign Keys:** `customer_dim_id`, `store_dim_id`, `date_dim_id`, `staff_dim_id`.

3.3 Dimension Tables

Dimension Name	Core Attributes	Source OLTP Tables	Analytical Purpose
Dim_Customer	customer_id, first_name, last_name, email, active_status, city, country	customer, address, city, country	Segments demographic trends, tracks consumer acquisition, and identifies elite high-value cohorts.
Dim_Film	film_id, title, release_year, language, rental_duration, rental_rate, length, rating, category_name	film, language, film_category, category	Evaluates inventory popularity based on movie rating, production year, and target film genre.
Dim_Store	store_id, manager_name, store_city, store_country	store, staff, address, city, country	Provides geographical aggregation to benchmark regional performance across different branches.
Dim_Staff	staff_id, first_name, last_name, store_id, active	staff	Measures clerk productivity and correlates personnel availability with transactional volume.
Dim_Date	date_key, full_date, day_of_week, month, quarter, year	Generated via code	Serves as the absolute temporal coordinate to enable month-over-month and seasonal financial trend analysis.

4. Dimensional Model Diagram

The proposed structural architecture implements a high-performance **Star Schema** layout. The transactional fact tables (`Fact_Rental` and `Fact_Payment`) establish the central core of the environment, directly mapped to the peripheral denormalized dimensions in a standard One-to-Many (**1:∞**) structural relationship.

Denormalization Strategy: Relational paths found in the OLTP model have been intentionally flattened. For example, rather than forcing downstream queries to execute sequential multi-table joins across address, city, and country structures to identify customer residence, these attributes are collapsed into explicit, readable textual data fields directly within `Dim_Customer` and `Dim_Store` . This eliminates costly relational query compilation time and accelerates operational dashboards.

5. ETL Pipeline Design

The Extract, Transform, and Load (ETL) pipeline was designed and mapped utilizing Python and the Pandas data analysis library to handle intermediate structured transformations:

5.1 Extract

The ingestion engine initializes an active database connection to the production `Sakila` database instance via localized relational database adapters. Analytical source data blocks are safely read and extracted into operational system memory as structured Pandas DataFrames, preventing operational locks on active transaction layers.

5.2 Transform

- **Structural Consolidation & Scrubbing:** Disconnected lookups are mapped and flattened (such as joining `film` with `category` records). Null attributes, incomplete values, and formatting discrepancies are dynamically cleaned.
- **Surrogate Key Generation:** The transformation engine generates localized, auto-incrementing integer keys for every dimensional entity. This isolates the warehouse from any natural key updates occurring in upstream source business platforms.
- **Temporal Engineering:** Transactional timestamps are systematically parsed to extract separate explicit granular entities—such as day names, months, structural quarters, and calendar years—populating rows in `Dim_Date`.

5.3 Load

The targeted data warehouse schema is prepared, and dimensional lookups are loaded first to fully satisfy database constraints. Once dimension records are securely committed, data arrays and calculated metrics are pushed directly into the central fact tables.

6. Data Quality Considerations

To preserve analytical truth and prevent dirty or mismatched data from impacting metrics, the pipeline enforces strict validation checkpoints:

- **Surrogate Key Uniqueness:** Enforces primary key restrictions across all warehouse dimensions, completely preventing duplicate row entries that would otherwise artificially expand metric values during summary computations.
- **Referential Integrity Constraints:** Ensures that all incoming transactional rows contain foreign keys that exist in the dimensional tables before allowing record writes to the fact tables.
- **Value Inversion Safeguards:** Validates that financial fields (`amount`) evaluate strictly as positive numbers (`amount > 0`), and verifies that record return dates are chronologically sequential to their initial rental dates.

7. Conclusion

This dimensional design successfully transforms raw transactional logs into high-value analytical assets. Migrating from a normalized transactional system to a star schema structure removes technical reporting bottlenecks. Decision-makers can now run real-time queries to monitor branch profits, customer lifetimes, and inventory velocity. This closes the gap between raw data and smart marketing and operational adjustments, driving long-term revenue growth.